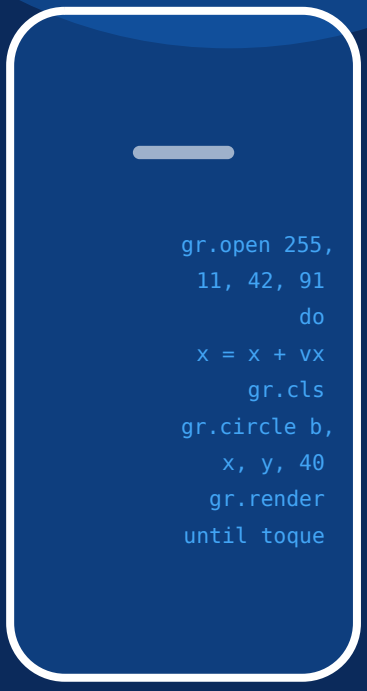
PROGRAMAÇÃO DE BOLSO RFO BASIC

AULA

06

Animação



Sua primeira animação, passo a passo.

RECURSO DO CELULAR

Tela gráfica

gráficos gr.* · coordenadas · animação · colisão

Projeto de Ensino em Programação · Novembro de 2026

NESTA AULA VOCÊ VAI

- ✓ Abrir a **tela gráfica** e desenhar um círculo com `gr.*`.
- ✓ Entender o **sistema de coordenadas** da tela, com o eixo y de ponta-cabeça.
- ✓ Criar **animação**: mudar a posição e redesenhar a cada volta do laço.
- ✓ Detectar **colisões** com as bordas e **inverter** a velocidade.
- ✓ Conhecer três jeitos de animar: `gr.cls`, `gr.modify` e `gr.move`.

Até aqui o celular falou, perguntou e calculou. Hoje ele vai **desenhar**, e a partir de uma única bolinha vamos subir cinco degraus: desenhá-la, mantê-la na tela, fazê-la atravessar a tela, bater e voltar, e por fim **rebater**. Cada degrau muda só um pouquinho do anterior, então o código novo cabe sempre numa ou duas linhas.

1 O Sistema de cores RGBA

O sistema de cores RGB (Red, Green, Blue, Alpha) é um modelo de cores aditivo baseado na emissão de luz, utilizado por telas de computadores, celulares e monitores para renderizar imagens e interfaces gráficas.

As cores são formadas pela combinação de feixes de luz nas três cores primárias sobre uma tela originalmente preta (desligada). Cada uma das três cores possui um canal de intensidade que varia de 0 a 255.

O valor *alpha* é quantidade a transparência, ou seja, determina o quanto da cor do fundo irá atravessar o objeto desenhado.

Nível de Opacidade	Alfa (0-255)	Efeito Visual
Opaco (Padrão)	255	Oculta totalmente o fundo
Alta Opacidade	192	Sombra suave / vidro fosco escuro
Média Transparência	128	Mistura perfeita entre cor e fundo
Baixa Opacidade	64	Marca d’água / sobreposição leve
Invisível	0	Totalmente transparente

2 A tela gráfica

Até agora usamos `print`, que escreve no **console**. Para desenhar, o hBasic tem uma segunda tela, a **gráfica**, controlada por comandos que começam com `gr.`:

Comando	O que faz

`gr.open a, r, g, b`	Abre a tela gráfica com uma cor de fundo.
`gr.orientation 1`	Fixa a tela em **retrato** (`0` é paisagem).
`gr.close`	Fecha a tela gráfica.

Listagem 1: Abrir o modo gráfico, canvas

`tela.bas` `basic`

```
1  gr.open 255, 11, 42, 91
2  gr.orientation 1
3  pause 3000
4  gr.close
5  end
```

RODAR Execute. Será mostrada uma tela com o mesmo azul-escuro das capas deste material por 3 segundos.

Coordenadas

A origem das coordenadas na tela do celular fica no canto **superior** esquerdo. O eixo x cresce para a direita e o y para baixo. Não se preocupe se você confundir, isso é normal já que as coordenadas não estão distribuídas como aprendemos na matemática.

> **ATENÇÃO**
>
> Na tela, o eixo y está de **ponta-cabeça** em relação ao plano cartesiano que usamos na escola. Um ponto "mais baixo" tem `y` **maior**.

3 Figuras geométricas simples

É necessário determinar a cor do objeto antes de desenhá-lo e ele torna-se visível apenas após a renderização.

Comando	O que faz
`gr.color a, r, g, b`	Escolhe a cor.
`gr.render`	Atualiza a tela com tudo o que foi desenhado.

Em todos os exemplos a seguir, o parâmetro `obj` é uma variável que guarda a referência do objeto na memória. Essa referência é o **número do objeto**, isto é, a "etiqueta" e é útil para quando quisermos mexer nele depois.

Círculo

O comando abaixo cria um círculo com raio `radius` ao redor do centro (`x`, `y`).

```
1  Gr.circle obj, x, y, radius
```

O comando `gr.circle` recebe uma variável (`obj`) que guarda o **número do objeto**. É a "etiqueta" do círculo, útil quando quisermos mexer nele depois.

O programa a seguir desenha três circulos de diametro 80 pixels lado a lado com um espaçamento de 20 pixels entre eles.

Listagem 2: Desenho de três círculos com espaçamento uniforme

`circulos.bas` `basic`

```
1   gr.open
2
3   gr.color 255, 255, 0, 0, 1
4   gr.circle c1, 50, 100, 40
5
6   gr.color 255, 0, 255, 0, 1
7   gr.circle c2, 150, 100, 40
8
9   gr.color 255, 0, 0, 255, 1
10  gr.circle c3, 250, 100, 40
11
12  gr.render
13  pause 3000
14  gr.close
```

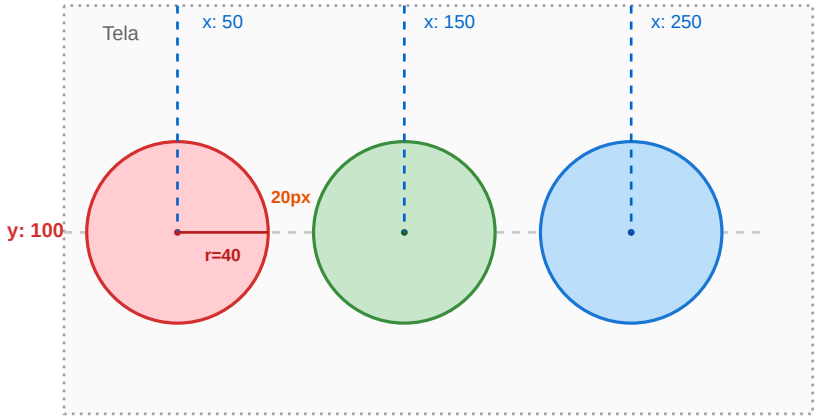


Figura 1: Geometria do comando `gr.circle`.

O comando `gr.open` sem valores significa o mesmo que `gr.open 255, 255, 255, 255`, ou seja, uma tela com fundo branco. Para cada retangulo, definimos a cor de preenchimento antes. O sistema não desenha o circulo quando definimos `gr.circle`, na verdade ele adiciona em uma lista chamada **Display List**. Somente quando o `gr.render` é chamado que o sistema desenha tudo dentro dessa lista especial.

Os objetos são desenhados na tela na ordem em que aparecem na *Display List*. Os objetos não ficam visíveis na tela até que o comando `gr.render` seja chamado.

Retângulo

```
1  gr.rect obj, left, top, right, bottom
```

Cria um retângulo delimitado pelas coordenadas informadas. Obviamente, `right - left` é a largura e `bottom - top` é a altura do retângulo.

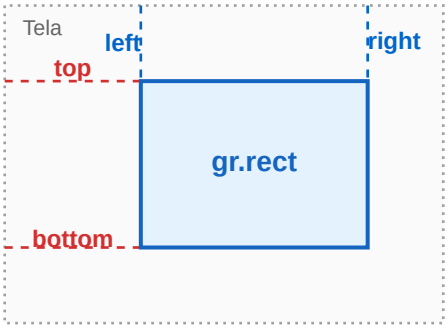


Figura 2: Entendimento das coordenadas delimitadoras no canvas.

O programa a seguir desenha três quadrados (80x80 pixels) lado a lado com um espaçamento de 20 pixels entre eles.

Listagem 3: Retangulos

`retangulos.bas` `basic`

```
1   gr.open
2
3   gr.color 255, 255, 0, 0, 1
4   gr.rect r1, 50, 100, 130, 180
5
6   gr.color 255, 0, 255, 0, 1
7   gr.rect r2, 150, 100, 230, 180
8
9   gr.color 255, 0, 0, 255, 1
10  gr.rect r3, 250, 100, 330, 180
11
12  gr.render
13  pause 3000
14  gr.close
```

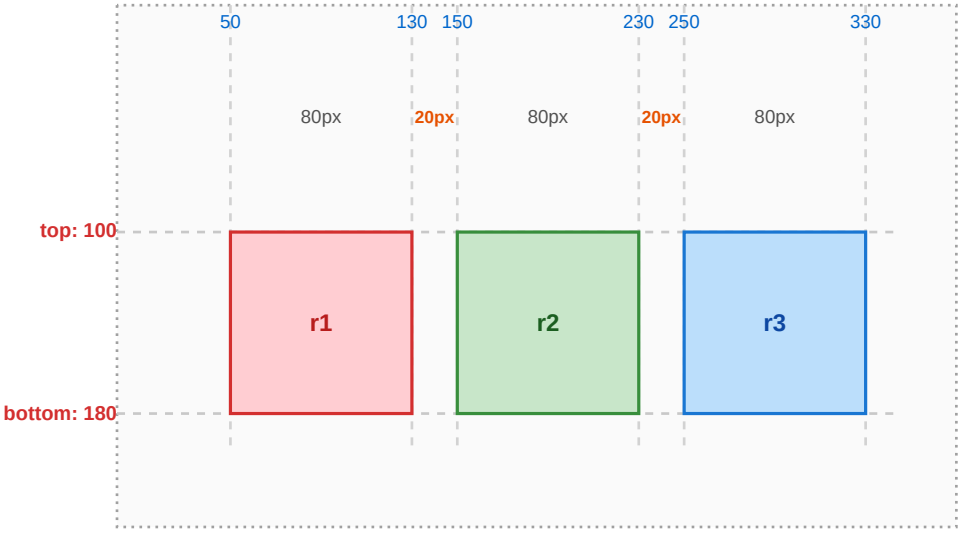


Figura 3: Distribuição horizontal de três retângulos idênticos.

DESAFIO

Crie um programa que desenhe três quadrados idênticos na horizontal alinhados e igualmente espaçados.

Bônus 1: use um `for`

Outros objetos gráficos

Comando	Efeito
`gr.point obj, x, y`	Cria um ponto na coordenada indicada.
`gr.line obj, x1, y1, x2, y2`	Cria uma linha entre os pontos (`x1`, `y1`) e (`x2`, `y2`)

4 Animação

Nesta seção iremos abordar um pouco de animação de nossos objetos criados no hBasic.

Passo 1: desenhar

Primeiro, vamos desenhar um circulo no centro da tela. O comando `gr.screen w, h` devolve a largura (`w`) e a altura (`h`) da tela, em pixels. Logo, o centro deve estar no ponto `(w/2,h/2)`

Listagem 4: Centro da tela

`circulo-centro.bas` `basic`

```
1   gr.open 255, 11, 42, 91
2   gr.orientation 1
3   gr.screen w, h
4
5   gr.color 255, 66, 165, 245
6   gr.circle bolha, w / 2, h / 2, 60
7
8   gr.render
9   pause 3000
10  gr.close
11  end
```

RODAR Execute. Uma bolinha azul aparece no centro da tela por 3 segundos. Mude o raio, a cor e a posição.

ATENÇÃO

A tela gráfica só é atualizada quando executamos `gr.render`. Sem ele, você desenha "nos bastidores" e nada aparece. Lembre-se a *Display List*.

DESAFIO **Quadrado no centro**

Desenhe um quadrado no centro da tela.

Bônus: Desenhe uma matriz 3x3 de quadrados igualmente espaçados.

Passo 2: deixar na tela

Três segundos é pouco. Queremos a bolinha ali **até que o usuário decida sair**. Um laço que espera um evento é a ferramenta certa. O evento agora é um **toque na tela**, detectado por `gr.touch`. Usando o código anterior, depois de `gr.render`:

Listagem 5: Detectar toque na tela

`toque.bas` `basic`

```
1  do
2    gr.touch toque, tx, ty
3    pause 50
4  until toque
5
6  gr.close
7  end
```

O comando `gr.touch toque, tx, ty` guarda em `toque` um valor **verdadeiro** se há um dedo na tela e, em `tx` e `ty`, o ponto tocado. O `do ... until toque` repete até acontecer um toque. O `pause 50` dá um respiro ao aparelho entre as verificações.

RODAR Execute. A bolinha fica na tela por quanto tempo você quiser. Toque para encerrar.

Passo 3: atravessar a tela

Agora a bolinha deve andar. Uma animação é uma **sequência de quadros**. Dessa forma, criaremos um laço e a cada volta mudamos a posição e redesenhamos a cena. A **velocidade** `vx` diz quantos pixels, na horizontal, a bolinha anda por quadro.

Listagem 6: Bolinha se deslocando pela tela

`circulo-move.bas` `basic`

```
1   gr.open 255, 11, 42, 91
2   gr.orientation 1
3   gr.screen w, h
4
5   raio = 40
6   x = raio
7   y = h / 2
8   vx = 6
9
10  while x - raio < w
11    x = x + vx
12
13    gr.cls
14    gr.color 255, 66, 165, 245
15    gr.circle bolha, x, y, raio
16    gr.render
17    pause 15
18  repeat
19
20  pause 1000
21  gr.close
22  end
```

Cada quadro faz três coisas, sempre nesta ordem:

1. **Atualiza** os números (`x = x + vx`).
2. **Apaga** o quadro anterior com `gr.cls`.
3. **Redesenha** a cena e mostra com `gr.render`.

CONCEITO · ANIMAÇÃO POR QUADROS

O `gr.cls` esvazia a tela, e o laço redesenha a bolinha na posição nova. Como fazemos isso muitas vezes por segundo, o olho enxerga **movimento contínuo**, como num desenho animado.

A bolinha sai de cena quando a sua borda **esquerda** (`x - raio`) passa da borda direita da tela (`w`). Por isso o laço roda enquanto `x - raio < w`.

RODAR Execute e observe a bolinha sumir pela direita. Aumente o `vx` para 15 e depois diminua para 2. O que muda no tempo da travessia?

DESAFIO **Movimento vertical**

Faça a bolinha descer verticalmente.

Bônus: Faça a bolinha descer em diagonal.

Passo 4: colisão

Em vez de sair, a bolinha deve **bater na borda e voltar**. O que acontece numa batida? A **velocidade troca de sinal**, logo se ela ia para a direita (`vx` positivo), passa a ir para a esquerda (`vx` negativo). Mas **quando** ela bate? Quando a sua borda toca a parede:

Parede	Condição de batida
Esquerda	`x < raio`
Direita	`x > w - raio`
Topo	`y < raio`
Base	`y > h - raio`

Estamos usando um circulo como modelo e se fosse, por exemplo, um quadrado?

DICA

No desenvolvimento prático de jogos, existe uma técnica chamada **Bounding Box**, também amplamente conhecida como **Hitbox**. Trata-se de uma simplificação geométrica onde, em vez de testar a colisão pixel por pixel da imagem do personagem (o que é computacionalmente pesado), envolve-se o *sprite* dentro de uma forma simples (como um retângulo) para realizar os cálculos matemáticos.

Listagem 7: Bolinha que rebate na tela

`circulo-bate.bas` `basic`

```
1   gr.open 255, 11, 42, 91
2   gr.orientation 1
3   gr.screen w, h
4
5   raio = 40
6   x = raio
7   y = h / 2
8   vx = 8
9
10  do
11    x = x + vx
12
13    if x > w - raio | x < raio
14      vx = -vx
15    endif
16
17    gr.cls
18    gr.color 255, 66, 165, 245
19    gr.circle bolha, x, y, raio
20    gr.render
21
22    gr.touch toque, tx, ty
23    pause 15
24  until toque
25
26  gr.close
27  end
```

Observe as linhas 13-15, são elas que determinam quando a bolinha deve mudar de direção.

Como o laço agora não tem fim natural, voltamos ao `do ... until toque` do Passo 2. O operador `|` significa **ou**: a bolinha inverte a velocidade se bater na direita **ou** na esquerda.

RODAR Execute e veja a bolinha indo e vindo. Mude o `vx` para −8. Ela ainda funciona? Por quê?

CONCEITO · REFLEXÃO

Inverter o sinal da velocidade é como a **reflexão** de uma bola numa parede lisa. É a regra mais simples de colisão que existe, e ela basta para muitos jogos.

A esta altura, podemos propor o próximo desafio.

DESAFIO

No alvo: craque do timing

1. Um circulo parado no centro com somente a borda desenhada.
2. Um circulo azul percorrendo a horizontal e passando por cima do circulo central.
3. Quando o usuário tocar na tela, o circulo deve parar.
4. O objetivo e conseguir parar exatamente no centro do circulo central.

Dica: se adicionarmos o valor 0 no final da lista de valores em `gr.color`, será preenchida somente a borda, se colocarmos 1 será preenchido somente o corpo.

Bonus: troque o centro central por um quadrado que caiba exatamente o circulo azul

5 Passo 5: rebater

A bolinha agora rebate nas **quatro** paredes. Precisamos só de uma velocidade vertical `vy` e de um segundo `if` para o topo e a base. O comportamento **vertical** e o **horizontal** são independentes, pois cada eixo tem sua posição, sua velocidade e seu `if`. O resultado, juntos, é o movimento diagonal que rebate.

Listagem 8: Bolinha que rebate em duas direções

`circulo-rebate.bas` `basic`

```
1   gr.open 255, 11, 42, 91
2   gr.orientation 1
3   gr.screen w, h
4
5   raio = 40
6   x = w / 2
7   y = h / 3
8   vx = 12
9   vy = 9
10
11  do
12    x = x + vx
13    y = y + vy
14
15    if x > w - raio | x < raio
16      vx = -vx
17    endif
18    if y > h - raio | y < raio
19      vy = -vy
20    endif
21
22    gr.cls
23    gr.color 255, 245, 158, 11
24    gr.circle bolha, x, y, raio
25    gr.render
26
27    gr.touch toque, tx, ty
28    pause 15
29  until toque
30
31  gr.close
32  end
```

Adicionamos um velocidade vertical `vy = 9` e um teste `if` se a bola tocou a parede de cima ou de baixo.

RODAR Execute e toque para sair. Mude `vx` e `vy` para direções e velocidades diferentes. A trajetória lembra uma bola de bilhar?

ATENÇÃO

Fizemos tudo no modo gráfico com `gr.cls`. Com ele, **apagamos tudo** e recriamos os objetos a cada quadro, como pintar uma tela nova a cada volta do laço.

6 Quebre o código!

Nos programas dos passos 3 a 5, mude **uma coisa de cada vez**:

QUEBRAR Apague o `gr.render` do laço. O que aparece na tela? Por quê?

QUEBRAR Apague o `gr.cls`. O que acontece com a imagem? E com a velocidade do programa depois de alguns segundos?

QUEBRAR Apague a linha `vx = -vx`. O que a bolinha faz ao chegar na parede?

QUEBRAR Aumente o `vx` para 80. A bolinha ainda "bate" na parede de forma convincente? Por que isso pode acontecer?

QUEBRAR No Passo 3, troque `x - raio < w` por `x < w`. Qual é a diferença visual no momento em que a bolinha some?

QUEBRAR Apague o `pause 15`. A animação fica mais rápida? Mais suave?

DICA

O quarto experimento revela uma limitação: a bolinha só "enxerga" a parede quando já está **dentro** dela. Se andar rápido demais, pode atravessá-la entre dois quadros. Jogos de verdade tratam disso com cuidado extra.

DESAFIO

Quique de verdade

Uma bola de verdade não rebate com velocidade constante, pois a **gravidade** a puxa para baixo, e cada batida faz perder um pouco de energia.

1. crie `g = 0.6` e, a cada quadro, faça `vy = vy + g` **antes** de atualizar `y`;
2. ao bater na base, coloque a bolinha **exatamente** em `y = h - raio` e inverta a velocidade perdendo energia: `vy = -vy * 0.85`; se `abs(vy) < 1` faça `vy = 0` para que ela pare;
3. deixe as paredes laterais como estão.

Pista para a base:

```
if y > h - raio
  y = h - raio
  vy = -vy * 0.85
  if abs(vy) < 1 then vy = 0
endif
```

Bônus 1: troque a cor da bolinha por uma cor **aleatória** a cada batida, com `rnd()`. Você vai precisar de três variáveis para guardar as componentes vermelho, verde e azul.

Bônus 2: ponha **duas** bolinhas, cada uma com suas variáveis (`x1`, `y1`, `vx1`, `vy1` e `x2`, `y2`, `vx2`, `vy2`). Repare quanto código se repete: isso é um convite para a próxima novidade do curso, as **funções**.

7 Resumo

Ideia ou comando	Para que serve
`gr.open` e `gr.close`	Abrem e fecham a tela gráfica.
`gr.screen w, h`	Descobre o tamanho da tela em pixels.
Origem no canto superior esquerdo	`y` cresce para baixo.
`gr.color`	Escolhe cor e estilo (contorno ou preenchido).
`gr.circle`	Desenha um círculo e guarda seu número em uma variável.
`gr.rect`	Desenha um retangulo e guarda seu número em uma variável.
`gr.render`	Atualiza a tela. Sem ele, nada muda.
`gr.touch t, tx, ty`	Detecta toque; ótimo para `do ... until`.
Animação por quadros	Atualizar, apagar, redesenhar e mostrar, repetidamente.
`gr.cls`	Apaga tudo; o quadro é redesenhado por inteiro a cada volta.

Velocidade `vx`, `vy`	Quantos pixels a bolinha anda por quadro em cada eixo.
Colisão com a parede	Condição de batida e inversão de sinal: `vx = -vx`.

NA PRÓXIMA AULA

Aula 7: Funções. E se, em vez de `vx` e `vy` fixos, quem movesse a bolinha fosse a **inclinação do celular**? Juntaremos o **acelerômetro** da Aula 5 com a bolinha desta aula, em degraus bem pequenos: primeiro só o eixo x, depois os dois eixos, e no fim um nível que avisa quando você está nivelado.